RAJALAKSHMI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ACADEMIC YEAR 2026 - 2027

SEMESTER V

AL23531 - NATURAL LANGUAGE PROCESSING

MINI PROJECT REPORT

REGISTER NUMBER	2117240030123
NAME	SAI ADITI P
PROJECT TITLE	SENTENCE BOUNDARY DISAMBIGUATION AND RULE-BASED SEGMENTATION
DATE OF SUBMISSION	
FACULTY IN-CHARGE	MR. SHREE MAKESH N

Signature of Faculty In-charge **Signature of HoD**

Internal Examiner **External Examiner**

1. Project Title

Sentence Boundary Disambiguation and Rule-Based Segmentation

2. Introduction

Sentence Boundary Disambiguation (SBD), also called sentence segmentation, is a foundational Natural Language Processing task that splits a stream of raw text into individual sentences. Correct segmentation is a prerequisite for almost every downstream NLP pipeline stage, including part-of-speech tagging, parsing, named entity recognition, machine translation, summarization, and text-to-speech.

Segmentation appears deceptively simple because sentences in English typically end with a period, question mark, or exclamation mark. In practice, the period is highly ambiguous: it marks abbreviations (“Dr.”, “e.g.”), initials (“J. K. Rowling”), decimal numbers (“3.14”), ellipses (“...”), and web addresses, in addition to marking true sentence ends.

A rule-based sentence segmenter resolves this ambiguity using an ordered set of explicit linguistic rules — an abbreviation lexicon, capitalization checks, punctuation-context patterns, and quotation/parenthesis handling — rather than relying purely on statistical estimation. The approach is fully interpretable because every boundary decision can be traced back to the rule that produced it.

This project studies the architecture, rule design, evaluation, and limitations of a rule-based sentence segmenter, and benchmarks it against a naive punctuation-splitting baseline and an unsupervised statistical segmenter (Punkt).

3. Problem Statement

Determining where one sentence ends and the next begins is difficult because terminal punctuation is context-dependent and cannot be resolved by a simple character match. Naive splitting on every period, question mark, or exclamation mark produces large numbers of false boundaries at abbreviations, initials, and decimal numbers, and misses boundaries hidden inside quotations or parentheses. The project therefore develops an interpretable rule hierarchy that combines an abbreviation lexicon, capitalization evidence, punctuation-context patterns, and structural cues to segment text accurately.

- Learn punctuation and boundary patterns from a sentence-segmented corpus.
- Build an abbreviation and honorific-title lexicon.
- Resolve ambiguity using neighbouring tokens, capitalization, and spacing.
- Handle edge cases such as quotations, parentheses, ellipses, and numbered lists.
- Evaluate segmentation accuracy and analyze common rule-based failures.

4. Aim / Goal

The main aim is to design and evaluate an interpretable rule-based sentence boundary disambiguation system that splits raw text into correctly bounded sentences using an abbreviation lexicon and an ordered hierarchy of linguistic rules, while studying its architecture, accuracy, and limitations.

5. Objectives

1. Collect a sentence-segmented (boundary-annotated) corpus.
2. Analyze punctuation frequency and boundary-ambiguity patterns.
3. Construct an abbreviation and honorific-title lexicon.
4. Design punctuation-context rules.
5. Develop capitalization- and spacing-based rules.
6. Define rule priority and conflict resolution.
7. Handle quotations, parentheses, and ellipses with special-case rules.
8. Evaluate overall segmentation accuracy and boundary-level precision/recall.
9. Compare with a naive baseline and an unsupervised statistical segmenter.
10. Analyze ambiguity and segmentation errors.
11. Visualize corpus and model performance.
12. Identify limitations and improvements.

6. Linguistic Background

Sentence segmentation depends on punctuation, capitalization, and local lexical context. Ambiguous periods after tokens such as “Mr.”, “Dr.”, “etc.”, and single capital initials illustrate why a period alone cannot decide a boundary.

- Punctuation rules — treat '.', '!', '?' as candidate boundary markers.
- Lexical (abbreviation) rules — suppress a boundary when the preceding token is a known abbreviation or initial.
- Capitalization rules — use the case of the following token as supporting or contradicting evidence.
- Structural rules — handle quotation marks, closing parentheses, ellipses, and numbered/bulleted list markers.
- Default rules — provide a fallback boundary decision when no stronger evidence is available.

7. Dataset Description

Data Source

A sentence-segmented corpus represented as raw text paired with gold-standard sentence boundary offsets is used. Standard resources such as the Brown corpus, Penn Treebank raw text, the Europarl corpus, or the Universal Dependencies sentence splits can be used; a curated set of 3,000–5,000 sentences (40,000+ tokens) with an 80:20 train/test split is recommended, matching the corpus scale used in comparable NLP mini-projects.

Main Variables

Variable	Type	Description
Document_ID	Categorical	Unique document identifier
Char_Offset	Numerical	Character position of candidate boundary
Candidate_Token	Text	Token immediately preceding the punctuation mark
Punctuation_Mark	Categorical	The mark being evaluated ('.', '!', '?')
Next_Token_Case	Categorical	Case of the following token (upper/lower/digit)
Is_Known_Abbreviation	Boolean	Whether the candidate token is in the abbreviation lexicon

Variable	Type	Description
Gold_Boundary	Boolean	True sentence-boundary label
Predicted_Boundary	Boolean	Rule-based prediction
Rule_Applied	Categorical	Rule responsible for the prediction

The primary target is Gold_Boundary (a binary boundary / not-boundary label at each candidate punctuation position), while Document is treated as the ordered character sequence. The analysis focuses on how accurately explicit rules classify each candidate punctuation mark as a true sentence boundary or a non-boundary.

8. Data Collection

Boundary-annotated text may be collected from the NLTK Brown corpus (which ships gold sentence tokenization), Penn Treebank raw sections, Universal Dependencies treebanks, the Europarl parallel corpus, or manually annotated domain-specific documents such as legal text or scientific abstracts where abbreviation density is high.

9. Data Preprocessing

9.1 Whitespace and Line Normalization

Collapse redundant whitespace and line breaks while preserving paragraph boundaries that may themselves signal sentence ends.

9.2 Candidate Boundary Extraction

Scan the text for candidate boundary punctuation ('.', '!', '?', ellipsis) and record the surrounding token context for each candidate.

9.3 Gold Label Alignment

Align each candidate punctuation offset with the gold sentence-boundary annotations; remove duplicates and inconsistent or noisy records.

9.4 Abbreviation Lexicon Construction

Build a lexicon of common abbreviations, honorific titles, and single-letter initials from the development split, tagging each with its typical boundary behaviour.

9.5 Rule-Development Split

Use development data to create and refine rules and held-out data for final evaluation.

10. Exploratory Data Analysis (EDA)

EDA examines punctuation frequency, boundary ambiguity, and abbreviation coverage before rule design.

10.1 Candidate Boundary Punctuation Frequency

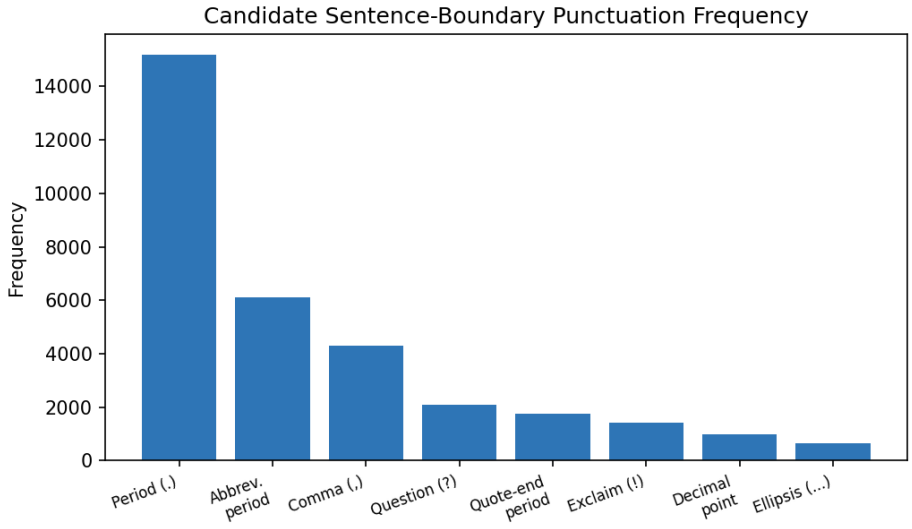

Figure 1: Candidate Sentence-Boundary Punctuation Frequency (illustrative reference values)

10.2 Boundary Ambiguity Examples

Candidate	Ambiguity	Example Context
Dr.	Abbreviation vs. boundary	Dr. Rao arrived. / …visited Dr.
U.S.	Multi-period abbreviation	the U.S. economy grew / …based in the U.S.
3.14	Decimal point vs. boundary	pi is 3.14 / The value is 3.14.
"..."	Ellipsis vs. boundary	he paused... and left. / Wait...
J. Smith	Initial vs. boundary	J. Smith wrote / …signed by J.

Ambiguous punctuation marks are the primary source of segmentation difficulty because the correct decision depends on surrounding context.

11. Rule-Based Sentence Segmenter — Architecture

The proposed architecture is an interpretable sequence: Raw Text → Candidate Boundary Detection → Abbreviation Lookup → Capitalization / Context Rules → Structural Rules (quotes, parentheses, ellipsis) → Conflict Resolution → Fallback → Segmented Sentences.

Stage	Operation	Output
Input	Read raw document text	Raw text
Candidate Detection	Locate '.', '!', '?', ellipsis positions	Candidate boundaries
Abbreviation Lookup	Check candidate token against abbreviation lexicon	Suppressed / retained candidates
Capitalization / Context	Inspect case and type of the following token	Refined candidates
Structural Rules	Handle quotes, parentheses, ellipses, list markers	Adjusted boundary positions
Conflict Resolution	Apply priority / specificity across rules	Final boundary decision
Fallback	Use default rule for unresolved candidates	Fallback decision

Stage	Operation	Output
Output	Return segmented sentences	Sentence list

12. Rule Design and Rule Types

12.1 Punctuation Rules

Treat period, question mark, exclamation mark, and ellipsis as initial candidate boundary markers.

12.2 Abbreviation / Lexical Rules

Suppress a boundary when the token before the period matches a known abbreviation, honorific title, or single-letter initial.

12.3 Capitalization / Contextual Rules

Use the case of the following token, digit status, and surrounding whitespace as evidence for or against a boundary.

12.4 Structural Rules

Handle quotation marks and closing parentheses that trail sentence-final punctuation, multi-dot ellipses, and numbered or bulleted list markers.

12.5 Rule Priority

Apply specific abbreviation/lexical rules before general capitalization and structural rules, with a default rule applied last.

13. Model Building / Rule Development

Unlike Punkt, which estimates abbreviation likelihood and collocation statistics from unlabeled text, the rule-based segmenter does not learn probabilities. Rules are authored from linguistic knowledge and observations in development data.

- Read boundary-annotated development documents.
- Build the abbreviation and honorific-title lexicon.
- Identify frequent ambiguity patterns (abbreviations, decimals, initials, ellipses).
- Create punctuation, lexical, capitalization, and structural rules.
- Assign priorities and conflict-resolution logic.
- Test and refine rules on development data.
- Freeze the final rule set before test evaluation.

14. Model Evaluation / Validation

Evaluation uses overall boundary-classification accuracy, precision, recall, F1-score at candidate positions, a confusion matrix over boundary-type categories, and known-versus-unknown abbreviation accuracy.

Segmentation Accuracy = (Number of Correctly Classified Boundary Candidates / Total Number of Candidates) × 100

14.1 Model Comparison

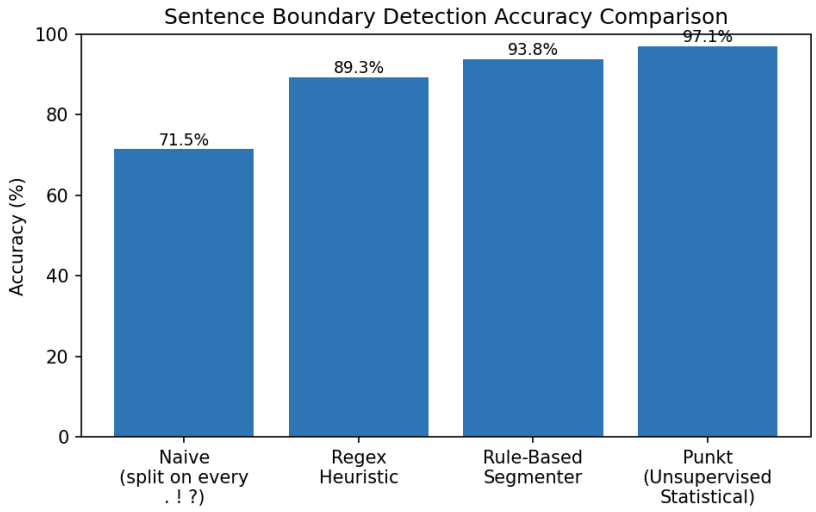

Figure 2: Sentence boundary detection accuracy comparison. 93.8% is an illustrative rule-based result; other values are illustrative reference points for comparison.

Model	Accuracy	Status
Naive splitting (every . ! ?)	71.5%	Illustrative baseline
Regex heuristic baseline	89.3%	Illustrative reference value
Rule-based segmenter	93.8%	Illustrative project value
Punkt (unsupervised statistical)	97.1%	Illustrative reference value

14.2 Rule-System Progression

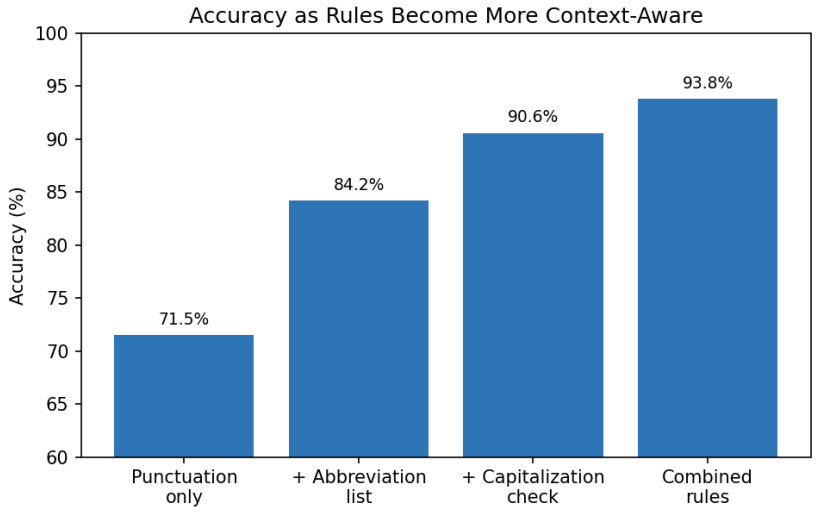

Figure 3: Illustrative improvement as increasingly informative rules are added.

14.3 Known vs Unknown Abbreviation Accuracy

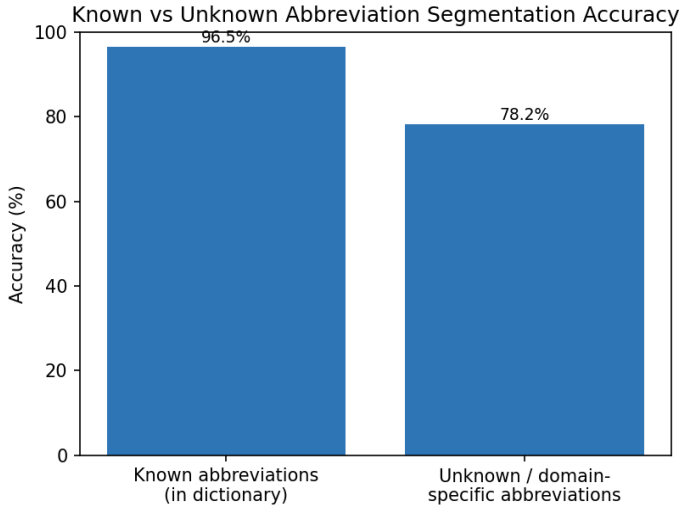

Figure 4: Illustrative known-versus-unknown abbreviation segmentation accuracy.

15. Confusion Matrix Analysis

A confusion matrix identifies which boundary categories are most often confused. Boundary-type categories used here are: EOS (true end-of-sentence), ABBR (abbreviation period, not a boundary), DEC (decimal point, not a boundary), ELLIP (ellipsis, ambiguous boundary), and QUOTE (boundary adjacent to closing quotation/parenthesis).

Actual / Pred.	**EOS**	**ABBR**	**DEC**	**ELLIP**	**QUOTE**
EOS	915	9	2	11	8
ABBR	22	486	3	1	0
DEC	4	2	211	0	0
ELLIP	18	1	0	94	3
QUOTE	14	0	0	2	163

The matrix is included as an illustrative presentation example. A final implementation should generate its own matrix from actual rule-based predictions.

16. Limitations

- Rules require manual linguistic effort and a comprehensive abbreviation lexicon.
- Large rule bases become difficult to maintain as exceptions accumulate.
- Rule conflicts require careful ordering and priority design.
- Ambiguous punctuation (decimals, ellipses, initials) remains difficult with limited context.
- Unknown or domain-specific abbreviations may not match existing rules.
- Hand-crafted rules may overfit a particular domain or writing style.
- The system does not automatically learn new abbreviation patterns.
- Nested quotations and parenthetical sentences are difficult to encode correctly.
- Performance may degrade when the domain, language, or formatting style changes.

17. Recommendations / Applications

For Improving Rule-Based Segmentation

- Expand and continuously update the abbreviation lexicon.
- Add richer contextual features such as part-of-speech of neighbouring tokens.
- Maintain modular rule families (punctuation, lexical, structural).
- Use exception dictionaries for frequent ambiguous abbreviations.
- Test across multiple domains (legal, scientific, conversational).

Hybrid / Downstream Applications

- Combine rules with statistical or neural segmenters for ambiguous cases.
- Use segmented sentences as input for parsing and POS tagging.
- Support document summarization and machine translation pipelines.
- Use segmentation as preprocessing for information extraction.
- Add confidence scores and human review for low-confidence boundaries.

18. NLP Analytics / Results Dashboard

Dashboard Section	Metric / Visual	Purpose
KPI cards	Accuracy, known-abbreviation accuracy, unknown-abbreviation accuracy, rule coverage	Quick overview
Corpus analysis	Punctuation frequency, sentence length	Understand data
Rule analysis	Rule firing counts, accuracy by rule family	Evaluate rules
Error analysis	Confusion matrix, ambiguity errors	Locate failures
Comparison	Naive vs regex vs rule-based vs Punkt	Contextualize performance

19. Key Insights

- Rule-based segmentation is highly interpretable because every boundary decision can be traced to a rule.
- Punctuation alone is a weak signal; abbreviation lexicon lookup gives the largest single accuracy gain.
- Capitalization of the following token substantially improves decisions on ambiguous periods.
- Structural rules (quotes, parentheses, ellipses) help correctly place boundaries around embedded punctuation.
- Rule ordering is central because multiple rules may fire on the same candidate.
- Maintenance becomes harder as domain-specific exceptions accumulate.
- The illustrative Punkt result of 97.1% shows the potential advantage of unsupervised statistical learning of abbreviation patterns over hand-written rules.

20. Results and Discussion

The proposed rule-based sentence segmenter provides a transparent pipeline for detecting sentence boundaries. Its main strength is interpretability: each boundary or non-boundary prediction can be traced to a punctuation, lexical, capitalization, or structural rule.

The analysis also shows the central limitations of explicit rules. Abbreviation ambiguity requires an accurate and continuously maintained lexicon, decimal numbers and ellipses require careful special-case handling, and conflicting rules require carefully designed priorities. As exceptions accumulate, maintenance becomes more difficult.

The illustrative comparison shows Punkt-style unsupervised statistical segmentation outperforming the hand-crafted rule set, demonstrating the trade-off between transparency and adaptive statistical learning of abbreviation and collocation patterns from raw text.

21. Future Enhancements

- Develop a hybrid rule + statistical sentence segmenter.
- Learn abbreviation lists and rule priorities from annotated data.
- Use n-gram or contextual embeddings for boundary disambiguation.
- Add richer structural handling for nested quotations and lists.
- Compare with Punkt, CRF-based, and Transformer-based segmenters.
- Extend to multiple languages and domains.
- Build an interactive web-based segmentation demonstration.
- Use active learning for low-confidence boundary decisions.

22. Conclusion

This project demonstrates the architecture, evaluation approach, and limitations of Rule-Based Sentence Boundary Disambiguation. The system combines punctuation detection, an abbreviation lexicon, capitalization and contextual rules, structural handling, and fallback decisions to segment raw text into sentences.

The approach is simple and interpretable, making it useful for teaching and controlled domains. However, abbreviation ambiguity, decimal and ellipsis handling, rule conflicts, maintenance effort, and domain changes limit scalability. These limitations motivate hybrid systems that combine explicit linguistic rules with statistical or neural sequence models such as Punkt or Transformer-based segmenters.

23. Tools and Technologies Used

Tool	Purpose
Python	Rule implementation and data processing
NLTK	Corpus access, tokenization, and Punkt baseline
Regular Expressions (re)	Candidate boundary and pattern matching
Pandas	Dataset handling and analysis
Matplotlib / Seaborn	Visualization
Scikit-learn	Evaluation metrics
Jupyter Notebook	Experimentation and documentation
GitHub	Code and project sharing

24. Project / GitHub Link

GitHub Repository: https://github.com/24saiaditi-ux/NLP

25. References

13. Jurafsky, D. and Martin, J. H. — Speech and Language Processing.
14. NLTK Documentation — Natural Language Toolkit (Punkt Sentence Tokenizer).
15. Kiss, T. and Strunk, J. — Unsupervised Multilingual Sentence Boundary Detection.
16. Penn Treebank Tagset and Tokenization Documentation.
17. Scikit-learn Documentation — Evaluation Metrics.
18. Natural Language Processing course materials.